TASKFLOW-PRO

DevOps & Cloud Infrastructure Documentation

AI-inspired team project management platform — architecture, CI/CD pipeline, and operations reference

Document type	DevOps / Infrastructure Reference
Cloud region	ap-south-1 (Asia Pacific — Mumbai)
Primary stack	Spring Boot microservices · React · AWS ECS Fargate
IaC tool	Terraform (>= 1.15, AWS provider ~> 6.0)
CI/CD	Jenkins
Version	1.0
Date	23 August 2026

Table of Contents

1. Executive Summary 3

2. System Architecture Overview 3

3. Technology Stack 4

4. Microservices Overview 5

5. AWS Infrastructure as Provisioned 5

5.1 Networking 5

5.2 Compute — ECS on Fargate 6

5.3 Load Balancing 6

5.4 Database 6

5.5 Container Registry (ECR) 6

5.6 Secrets Management 7

5.7 Logging & Monitoring 7

5.8 Not Yet Present in Terraform (vs. the Target Diagram) 7

6. CI/CD Pipeline (Jenkins) 7

6.1 What the Pipeline Does — and Doesn't — Automate 8

7. Containerization & Local Development 8

7.1 Docker Compose 8

7.2 Service Images 9

7.3 Frontend Development Workflow 9

8. Configuration & Environment Variables 9

9. Security Considerations & Recommendations 10

10. Deployment Runbook 11

10.1 Standard Release 11

10.2 Rollback 11

11. Appendix 12

11.1 Repository Structure (simplified) 12

11.2 Port Reference 12

11.3 Key Files 12

1. Executive Summary

TaskFlow-Pro is a Jira/Trello/Notion-inspired team project management platform, built as a set of Spring Boot microservices behind an API gateway, with a React single-page frontend. The backend is containerized with Docker and deployed to **AWS ECS on Fargate**, provisioned through **Terraform**, with a **Jenkins** pipeline handling build, test, and image publishing to Amazon ECR.

This document is the DevOps and infrastructure reference for the project. It covers:

- The system's architecture, both as designed (Section 2) and as actually provisioned in the Terraform codebase today (Section 5) — the two are not yet identical, and the gaps are called out explicitly rather than glossed over.
- The technology stack and per-service responsibilities.
- The CI/CD pipeline: what it automates, and what still requires a manual step.
- Local development via Docker Compose.
- Security posture and concrete recommendations, ranked by what matters most before this goes past a personal/demo deployment.
- An operational runbook for deploying, verifying, and rolling back a release.

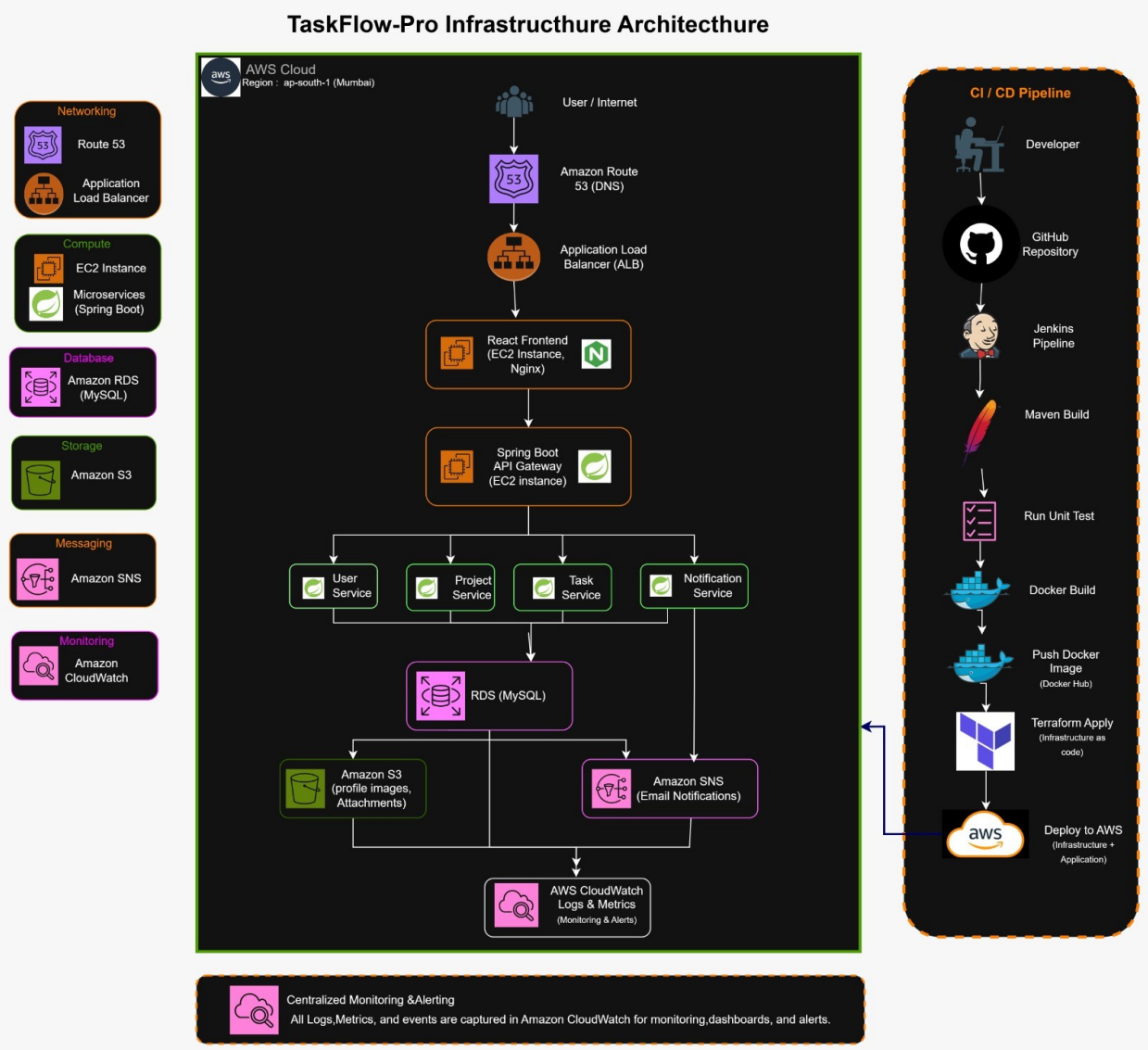
The intent is that a new engineer — or the project owner in six months — can read this and understand exactly how the system is built, shipped, and run, without having to reverse-engineer it from the Terraform state file.

2. System Architecture Overview

At a high level, a request flows through the platform as follows:

1. A user's browser resolves the application domain via **Amazon Route 53** and reaches the **Application Load Balancer (ALB)** over HTTP/HTTPS.
2. The ALB routes traffic to the **React frontend** (served via Nginx) and the **Spring Boot API Gateway**, which is the single entry point into the backend.
3. The API Gateway authenticates requests (JWT) and routes them, via **Eureka service discovery**, to one of four microservices: **User**, **Project**, **Task**, and **Notification** service.
4. Each microservice reads and writes to a shared **Amazon RDS for MySQL** instance. File assets (profile images, attachments) are intended to live in **Amazon S3**, and outbound email notifications are intended to go through **Amazon SNS**.
5. Logs and metrics from every component are collected in **Amazon CloudWatch** for monitoring and alerting.
6. Separately, a **Jenkins** pipeline builds each service from source, runs unit tests, builds Docker images, and pushes them to **Amazon ECR**; infrastructure changes are applied with **Terraform**.

The diagram below is the reference architecture for the platform:



NOTE *This diagram describes the platform's target architecture. Section 5 ("AWS Infrastructure as Provisioned") audits it line-by-line against what is actually defined in terraform/main.tf today, and flags every place the two currently diverge — most notably: no Route 53, S3, or SNS resources exist yet, and the ALB currently routes only to the API Gateway (there is no live target group for the React frontend).*

3. Technology Stack

Layer	Technology
Frontend	React 19, React Router 7, Axios, Vite 8 (dev server / bundler), ESLint
Backend services	Java 21, Spring Boot 3.5.4, Spring Cloud 2025.0.3
API Gateway	Spring Cloud Gateway (WebMVC variant), Eureka client, Spring Security
Service discovery	Netflix Eureka (discovery-server)
Auth	JWT via jjwt 0.12.x, Spring Security
API docs	springdoc-openapi (Swagger UI) on user-service
Database	Amazon RDS for MySQL 8.0

Layer	Technology
Object storage (planned)	Amazon S3 — AWS SDK v2 already included in user-service's pom.xml
Notifications (planned)	Amazon SNS
Containerization	Docker, multi-stage builds (maven:3.9-eclipse-temurin-21 → eclipse-temurin:21-jre)
Orchestration	AWS ECS on Fargate (awsvpc networking, 256 CPU / 512 MB per task)
Load balancing	AWS Application Load Balancer
IaC	Terraform >= 1.15.0, hashicorp/aws provider ~> 6.0
CI/CD	Jenkins declarative pipeline (Jenkinsfile), Windows batch ("bat") steps
Image registry	Amazon ECR (image scanning on push)
Secrets	AWS Secrets Manager
Logging	Amazon CloudWatch Logs (per-service log groups, 7-day retention)

4. Microservices Overview

Service	Local port	ECS port	Responsibility
discovery-server	8761	8761	Netflix Eureka registry — every other service registers here for discovery.
config-server	—	—	Spring Cloud Config (present in backend/config-server). Not yet deployed as an ECS service — see Section 5.8.
api-gateway	8080	8080	Single entry point; JWT validation and request routing to services via Eureka.
user-service	8081	8080	Accounts, authentication, profiles; pulls in the AWS S3 SDK for profile images.
project-service	8082	8080	Projects / boards.
task-service	8083	8080	Tasks within projects.
notification-service	8084	8080	Notifications; intended to publish via SNS.

NOTE *Local docker-compose gives every service a distinct host port; in ECS, each Fargate task gets its own elastic network interface (awsvpc mode), so every service can independently listen on container port 8080 (8761 for discovery-server) without colliding.*

5. AWS Infrastructure as Provisioned

Everything in this section is drawn directly from terraform/main.tf — it describes what actually gets created when someone runs terraform apply against this codebase, in the ap-south-1 (Mumbai) region, as a single flat configuration (no modules or workspaces).

5.1 Networking

- **VPC:** 10.0.0.0/16 (taskflow_vpc), DNS support and DNS hostnames enabled.

- **Public subnets:** 10.0.1.0/24 (ap-south-1a) and 10.0.2.0/24 (ap-south-1b) — host the ALB.
- **Private subnets:** 10.0.3.0/24 (ap-south-1a) and 10.0.4.0/24 (ap-south-1b) — host all ECS tasks and RDS.
- **Internet Gateway** + a public route table (0.0.0.0/0 → IGW) for the public subnets.
- **One NAT Gateway** (in public_1) + a private route table (0.0.0.0/0 → NAT) so private-subnet tasks can reach the internet (e.g. to pull images, call AWS APIs). This is a single point of failure for private-subnet egress — it lives in one AZ only.
- **Security groups:** ALB SG allows 80/443 from anywhere; ECS SG allows 8080 from the ALB SG plus 8080/8761 from the VPC CIDR; RDS SG allows 3306 only from the ECS SG. This is a sound least-privilege chain.

5.2 Compute — ECS on Fargate

Cluster `taskflow-pro-cluster` runs six Fargate services — user-service, project-service, task-service, notification-service, discovery-server, and api-gateway. Every task definition uses the same shape:

- awsvpc network mode, FARGATE launch type, private subnets, assign_public_ip = false
- 256 CPU units / 512 MB memory (0.25 vCPU / 0.5 GB) per task
- desired_count = 1 — no redundancy and no autoscaling configured for any service
- Execution role `taskflow-ecs-task-execution-role` (AmazonECSTaskExecutionRolePolicy) plus an inline policy granting `secretsmanager:GetSecretValue` scoped to the database secret

NOTE *user-service, project-service, and task-service pull DB_HOST/DB_PORT/DB_NAME as plain environment values and DB_USERNAME/DB_PASSWORD from Secrets Manager via the container definition's `secrets` block — the right pattern for credentials. notification-service's task definition, however, has no environment or secrets block at all in main.tf, so its DB/SNS configuration isn't wired up through Terraform yet.*

5.3 Load Balancing

- One internet-facing Application Load Balancer (taskflow-pro-alb) spanning both public subnets.
- One target group (taskflow-gateway-tg): HTTP:8080, target_type=ip, health check on GET /actuator/health (2 healthy / 3 unhealthy, 30s interval).
- One HTTP:80 listener, whose default (and only) action forwards all traffic to that target group.
- Only api-gateway is registered with the ALB. There is no second listener rule, path pattern, or target group for a React frontend service in this Terraform.

NOTE *This is the clearest gap versus the architecture diagram: today, the ALB is effectively "API Gateway only." The React frontend isn't served through the ALB at all yet — see Section 5.8 and Section 7.*

5.4 Database

- `aws_db_instance.taskflow` — MySQL 8.0 on db.t3.micro, 20 GB gp3 storage (auto-scales to 50 GB).
- Private only (publicly_accessible = false), single-AZ (multi_az = false), inside a DB subnet group covering both private subnets.
- **backup_retention_period = 0** (automated backups disabled), **skip_final_snapshot = true**, **deletion_protection = false** — reasonable for iterating in development, risky if left this way once real data matters. See Section 9.

5.5 Container Registry (ECR)

Four repositories are Terraform-managed, each with image_scanning_configuration.scan_on_push = true:

- taskflow-user-service · taskflow-project-service · taskflow-task-service · taskflow-notification-service

NOTE *The Jenkinsfile also builds and pushes taskflow-api-gateway and taskflow-discovery-server images to ECR, but neither has a matching aws_ecr_repository resource in main.tf. Either those two repositories were created by hand outside of Terraform, or those pipeline pushes are relying on a repository that Terraform doesn't know about and can't reproduce from scratch. Recommend adding both to main.tf so the registry is fully IaC-managed.*

5.6 Secrets Management

A Secrets Manager secret, `taskflow/pro/database`, stores username, password, host, port, and database name as JSON, and is consumed by ECS through the container definition's `secrets` block rather than as a plaintext environment variable — good practice.

NOTE *The RDS master password itself, however, is hardcoded as a plaintext literal ("TaskFlowDB2026") directly in terraform/main.tf, in both aws_db_instance.password and the secret's JSON payload — and a matching comment in the file even flags "this password MUST match the existing RDS instance," suggesting it's been manually kept in sync at least once already. That value also persists unencrypted in terraform.tfstate / terraform.tfstate.backup on disk. See Section 9 for a fix.*

5.7 Logging & Monitoring

One CloudWatch Log Group per service (/ecs/taskflow/<service-name>), 7-day retention, using the awslogs driver with an "ecs" stream prefix — for user-service, project-service, task-service, notification-service, discovery-server, and api-gateway.

NOTE *No CloudWatch alarms, dashboards, or metric filters exist in Terraform yet. "AWS CloudWatch Logs & Metrics" in the architecture diagram is accurate for log collection today, but the "Alerts" half of it — the actual monitoring/alerting layer — still needs to be built (e.g. alarms on ECS CPU/memory, ALB 5xx rate, target group unhealthy-host count, RDS free storage).*

5.8 Not Yet Present in Terraform (vs. the Target Diagram)

For a straightforward before/after read, everything the diagram shows that isn't in main.tf today:

- Route 53 hosted zone / DNS records — the ALB is currently reached by its raw *.elb.amazonaws.com DNS name.
- An Amazon S3 bucket for profile images/attachments — user-service's pom.xml already depends on the AWS S3 SDK, so the application code is ready for this; the bucket itself just isn't provisioned.
- An Amazon SNS topic for email notifications.
- React frontend hosting (EC2 + Nginx per the diagram, or an S3/CloudFront static site as a lighter-weight alternative) — today the frontend runs only as a local Vite dev server that proxies to the ALB (Section 7).
- Auto scaling policies for any ECS service.
- HTTPS — the ALB listener is HTTP:80 only, with no ACM certificate.
- A dedicated target group / routing rule for the frontend, and a config-server ECS service (the code exists in backend/config-server but has no task definition or service in main.tf).

6. CI/CD Pipeline (Jenkins)

The pipeline is defined in a single Jenkinsfile at the repository root, running as a declarative pipeline with agent any and Windows batch (bat) steps — so it's tied to a Windows-based Jenkins agent with Docker Desktop, the AWS CLI, and Maven available locally.

#	Stage	What it does
1	Checkout	checkout scm — pulls the pipeline's configured branch.
2	Docker Verification	docker --version / docker info / docker images — sanity-checks the agent's Docker daemon.
3–9	Build & Test (×4 services)	For user-service, project-service, task-service, notification-service, and api-gateway: mvnw.cmd package -DskipTests then mvnw.cmd test, run per-service via dir(...).
10	Build Discovery Server	mvn.cmd package -DskipTests (no test stage for discovery-server).
11	Docker Build	Builds all six images (user, project, task, notification, api-gateway, discovery-server), each tagged :ci.
12	Verify Docker Images	docker images against each of the six tags, for visibility in the build log.
13	AWS Identity Check	aws sts get-caller-identity, using the aws-ecr-credentials Jenkins credential (withCredentials — not hardcoded).
14	ECR Login	aws ecr get-login-password piped into docker login against the ap-south-1 ECR registry.
15	Tag & Push to ECR	All six images are tagged <repo>:ci and pushed to their respective ECR repositories.
16	Verify ECR Images	aws ecr describe-images for each repository, printed as a table.

6.1 What the pipeline does — and doesn't — automate

- **It builds, tests, and publishes images. It does not deploy them.** The pipeline's own success message ends with “NEXT STEP: Deploy infrastructure using Terraform” — applying Terraform and rolling ECS services onto the new image is a separate, manual step today.
- **Every image is pushed under one mutable tag:** `:ci`. Each run overwrites the previous :ci image in every repository. There's no git-SHA or build-number tag, so there's no way to look at a running ECS task and know exactly which commit produced its image, and no way to redeploy “the build from two days ago” once a newer :ci has been pushed.
- **Task Service integration tests are explicitly skipped** — a comment in the Jenkinsfile notes they depend on other microservices and Eureka being up, which the CI agent doesn't run.
- Credentials are handled correctly via a Jenkins credential binding (aws-ecr-credentials), not embedded in the Jenkinsfile.

7. Containerization & Local Development

7.1 Docker Compose

Two separate docker-compose.yml files exist in the repository:

File	Services	Notes
docker-compose.yml (root)	mysql, user-service, project-service, task-service	No eureka-server, notification-service, or api-gateway. Uses SPRING_DATASOURCE_* env var names. Looks like an earlier/partial version of the stack.

File	Services	Notes
backend/docker-compose.yml	mysql, eureka-server, user-service, project-service, task-service, notification-service, api-gateway	The complete stack, with correct service startup ordering via depends_on / condition: service_healthy. Uses DB_URL / DB_USERNAME / DB_PASSWORD / EUREKA_URL / JWT_SECRET env var names.

NOTE *The two files use different environment-variable names for the same settings (SPRING_DATASOURCE_URL vs DB_URL, for example), and only one of them actually brings up the full stack. Recommend keeping backend/docker-compose.yml as the single source of truth for local development and removing or clearly deprecating the root-level one, so there's no ambiguity about which command actually starts the app.*

7.2 Service Images

Every Spring Boot service uses the same two-stage Dockerfile pattern:

7. Build stage — maven:3.9-eclipse-temurin-21, copies the pom.xml first to cache dependency resolution (mvn dependency:go-offline in user-service's Dockerfile), then the source, and runs mvn clean package -DskipTests.
8. Runtime stage — eclipse-temurin:21-jre, copies only the built jar from the build stage, exposes the service's port, and runs it as java -jar app.jar.

NOTE *This produces small, reasonably clean runtime images. Two low-effort hardening steps worth adding: run the JRE stage as a non-root user, and pin base image digests (or at least minor versions) rather than floating tags, so a build today and a build in six months use the same base layer.*

7.3 Frontend Development Workflow

The React app runs via the Vite dev server on port 5173. Its `vite.config.js` proxies any request to `/api/*` to a target URL — defaulting to `http://localhost:8080` (matching docker-compose's api-gateway), or overridable via `VITE_API_PROXY_TARGET` in `frontend/.env.local` to point at the deployed ALB instead.

NOTE *This proxy setup is the fix for a CORS bug closed on 2026-08-20: axios.js previously had an absolute baseURL pointing straight at the ALB, and authApi.js kept its own separate proxied instance — so Login/Register worked (they went through the proxy) while Projects/Tasks failed with genuine cross-origin errors (they bypassed it). All frontend API modules now share one axios instance in frontend/src/api/axios.js with a relative baseURL of "/api", so every request goes through the Vite proxy in dev. The backend's own CORS config (SecurityConfig.java, allowing http://localhost:5173) was never the problem and didn't need to change.*

Because there's no S3/CloudFront or EC2+Nginx frontend deployment in Terraform yet (Section 5.8), this dev-proxy workflow is currently the only way the frontend is served — there is no deployed, publicly reachable frontend today.

8. Configuration & Environment Variables

Root-level `.env.example` documents the variables an operator needs to supply:

Variable	Purpose
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	AWS credentials (used locally / by tooling that needs direct AWS API access).

Variable	Purpose
DB_USERNAME / DB_PASSWORD	MySQL credentials for local/dev use.
JWT_SECRET	Signing secret for JWTs issued by user-service and validated by api-gateway.

The two docker-compose files diverge on naming (see Section 7.1): SPRING_DATASOURCE_URL/USERNAME/PASSWORD in the root file vs. DB_URL/DB_USERNAME/DB_PASSWORD/EUREKA_URL/JWT_SECRET in backend/docker-compose.yml. On AWS, ECS task definitions instead source DB_HOST/DB_PORT/DB_NAME as plain values and DB_USERNAME/DB_PASSWORD from Secrets Manager (Section 5.6) — so the same three services use three different variable-naming conventions across three environments (root compose, backend compose, ECS). Standardizing on one naming convention across all three would remove a recurring source of "why isn't this service picking up the DB connection" debugging.

Frontend: `VITE_API_PROXY_TARGET` in `frontend/.env.local` (not committed) — switches the Vite dev proxy between the local api-gateway and the deployed ALB.

9. Security Considerations & Recommendations

These are listed roughly in order of what's worth fixing first. None of them are unusual for a project at this stage — they're exactly the kind of thing a DevOps review is meant to surface before the workload carries anything that matters.

Finding	Risk	Recommendation
RDS master password is a plaintext literal in terraform/main.tf (and therefore in terraform.tfstate)	Anyone with read access to the repo or the state file has the database password	Generate it with a random_password resource, or supply it via a sensitive Terraform variable / CI secret — never as a literal in .tf source
terraform.tfstate and terraform.tfstate.backup are present in the local terraform/ folder	State files contain resource attributes (including the DB password above) in plaintext	Move to a remote backend (S3 + DynamoDB lock table) and confirm *.tfstate* is git-ignored
RDS: backup_retention_period = 0, deletion_protection = false	No point-in-time recovery; the database can be deleted with a single terraform destroy or console click	Enable automated backups (7+ days) and deletion protection once the data matters
Single NAT Gateway, single-AZ RDS, desired_count = 1 on every ECS service	No redundancy — one AZ outage or one crashed task causes a service outage	Acceptable for a personal/demo deployment; before production, add a second NAT, Multi-AZ RDS, and desired_count ≥ 2 per service
ALB listener is HTTP:80 only — no ACM certificate / HTTPS listener	Traffic between users and the ALB is unencrypted	Request an ACM certificate, add an HTTPS:443 listener, and redirect 80 → 443
No aws_ecr_repository resource for api-gateway or discovery-server, though Jenkins pushes images there	Infrastructure drift — part of the registry exists outside Terraform's knowledge	Add the two missing aws_ecr_repository resources to main.tf
Every CI image is pushed under one mutable tag, :ci	Can't trace which commit is running in ECS; a bad push silently overwrites the last good image	Tag images with the git commit SHA or Jenkins BUILD_NUMBER in addition to (or instead of) :ci
Six superseded ECS task-definition	Unclear which definition is	Treat Terraform as the single source of

Finding	Risk	Recommendation
JSON files live alongside the code in backend/api-gateway/ (ecs-v4.json, gateway-v4.json, gateway-v4-clean.json, gateway-v5-source.json, gateway-v5.json, gateway-v5-aws.json)	authoritative; risk of deploying a stale one by hand	truth for task definitions; delete or archive the superseded files outside the deploy path
A legacy gateway task-definition hardcodes the Eureka URL as a private IP (10.0.4.126) instead of a stable address	Fargate tasks get a new IP on every restart — if discovery-server restarts, api-gateway's Eureka registration can silently break, taking down all lb:// routing	Use AWS Cloud Map or a static internal NLB/DNS name for service discovery instead of a raw IP

10. Deployment Runbook

10.1 Standard Release

9. A developer pushes to the branch Jenkins watches (or triggers the job manually).
10. Jenkins checks out the source, builds and unit-tests each of the five Maven-based services with the Maven wrapper, and builds discovery-server without tests.
11. Jenkins builds all six Docker images, authenticates to AWS via the aws-ecr-credentials Jenkins credential, and pushes every image to its Amazon ECR repository tagged :ci.
12. Apply any infrastructure changes separately: from `terraform/`, run `terraform init`, `terraform plan`, then `terraform apply` against ap-south-1.
13. Because ECS services aren't wired to redeploy automatically when a new `:ci` image lands, force a new deployment per service, e.g.: `aws ecs update-service --cluster taskflow-pro-cluster --service <service-name> --force-new-deployment --region ap-south-1`
14. Verify the release: check RUNNING task count (ECS console or aws ecs describe-services), target group health for api-gateway in the ALB console, GET /actuator/health through the ALB's DNS name, and the relevant CloudWatch log group (/ecs/taskflow/<service-name>) for startup errors.

10.2 Rollback

Because every image is pushed under the single mutable `:ci` tag, the previous image is not retrievable once a new build has pushed over it — so today's rollback path is redeploying the **previous ECS task-definition revision**, not a differently-tagged image:

15. `aws ecs describe-task-definition --task-definition <family>` to confirm the current revision, then
16. `aws ecs update-service --cluster taskflow-pro-cluster --service <service-name> --task-definition <family>:<previous-revision>`

NOTE *Adopting immutable, commit-SHA image tags (Section 9) would let a rollback target a specific known-good image rather than relying on whichever task-definition revision happened to reference it.*

11. Appendix

11.1 Repository Structure (simplified)

Path	Contents
backend/api-gateway	Spring Cloud Gateway service; also holds several superseded ECS task-def JSON files (see Section 9)
backend/config-server	Spring Cloud Config server — not yet deployed to ECS
backend/discovery-server	Eureka registry
backend/notification-service	Notification microservice
backend/project-service	Project microservice
backend/task-service	Task microservice
backend/user-service	User microservice — includes taskdef.json and an RDS CA bundle (global-bundle.pem) for TLS DB connections
backend/docker-compose.yml	Full local stack (recommended source of truth — see Section 7.1)
frontend/	React + Vite single-page application
terraform/	All AWS infrastructure (main.tf, variables.tf, versions.tf, provider.tf)
Jenkinsfile	CI/CD pipeline definition
docker-compose.yml	Partial local stack (root level — see Section 7.1)
.env.example	Template for required environment variables

11.2 Port Reference

Service	Local (docker-compose)	AWS ECS (container port)
discovery-server	8761	8761
api-gateway	8080	8080 (behind ALB on port 80)
user-service	8081	8080
project-service	8082	8080
task-service	8083	8080
notification-service	8084	8080
MySQL	3307 (host) → 3306 (container)	3306 (RDS, private only)
Frontend (Vite dev server)	5173	not yet deployed

11.3 Key Files

File	Purpose
terraform/main.tf	All AWS infrastructure definitions
Jenkinsfile	CI/CD pipeline: build, test, Docker build, push to ECR

File	Purpose
backend/docker-compose.yml	Full local backend stack
frontend/vite.config.js	Dev server + /api proxy configuration
frontend/src/api/axios.js	Shared axios instance all frontend API modules must use
.env.example	Template for required environment variables